Advanced SQL

⏪ Recap

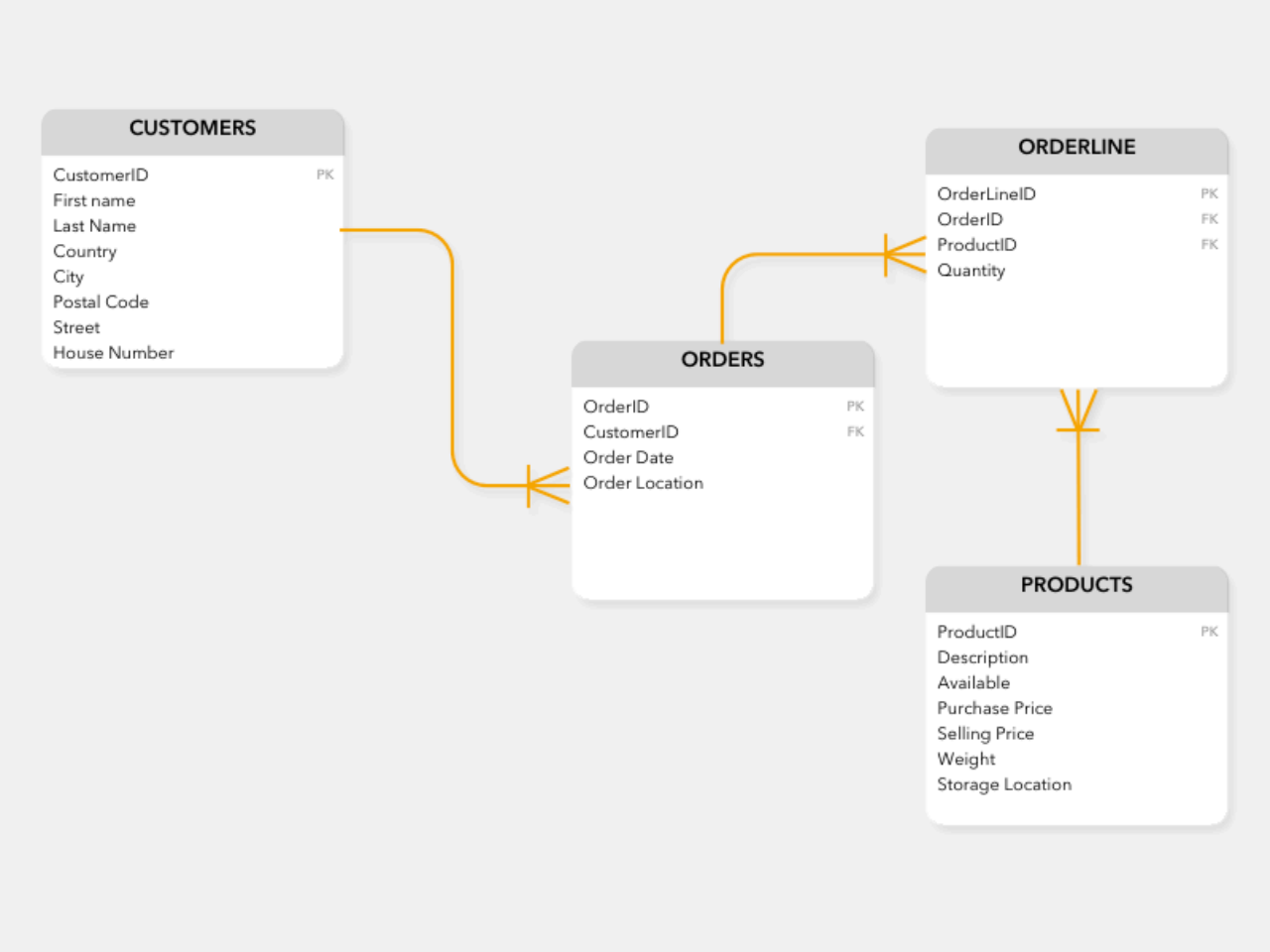
- `SELECT` columns `FROM` a table `WHERE` conditions are true
  - `LIKE` pattern matching operator (with `%` and `_`)
  - `IN` membership operator
  - `BETWEEN` range operator
- `LIMIT` the number of records returned
- `ORDER BY` columns

Sample ERD

Entities: Customers, Orders, Products, Orderline

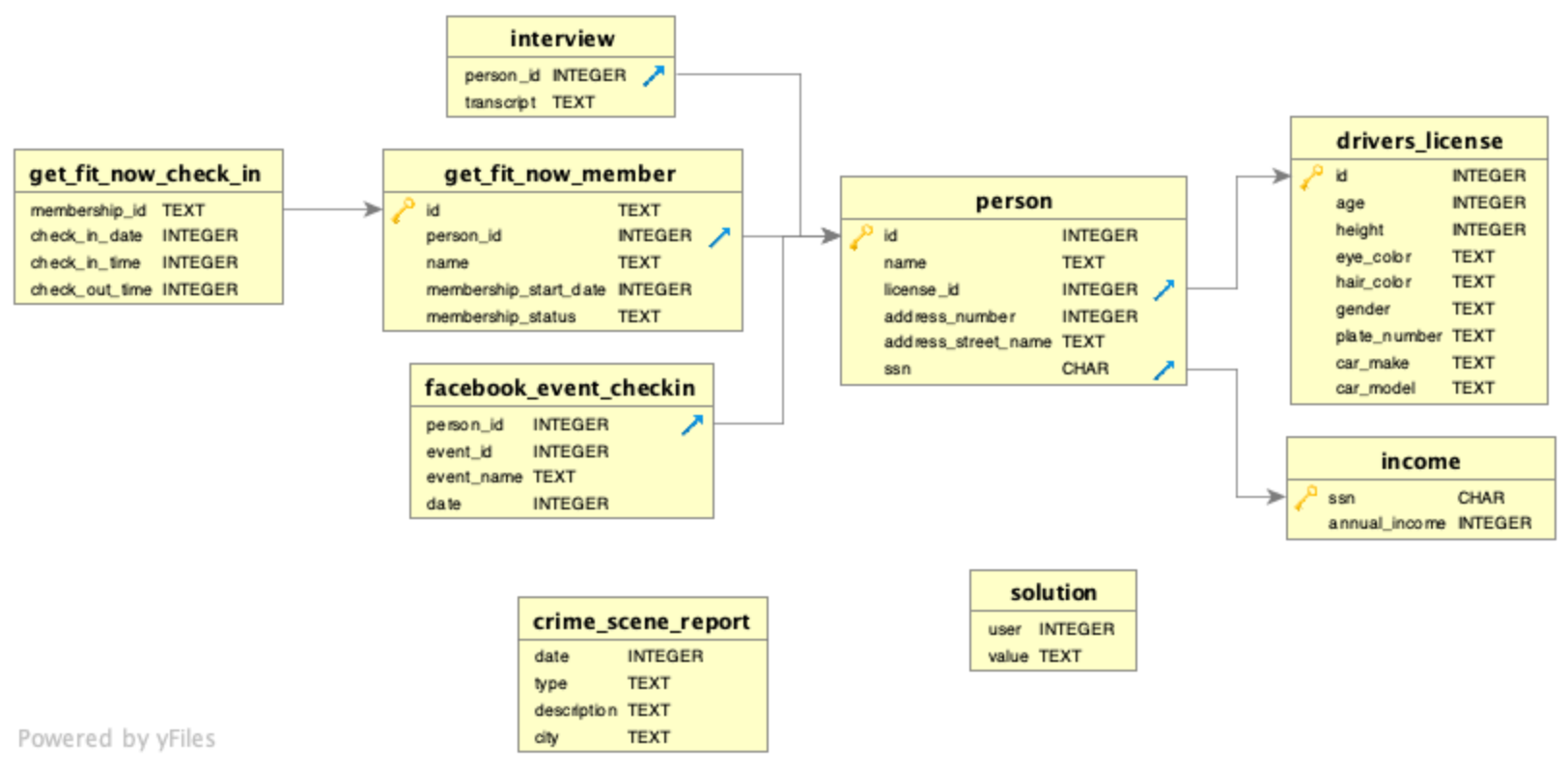
Relationships:

- Customers place Orders
- Orders contain Products via Orderline



⏪ Recap

- `get_fit_now_member` ➡️ `drivers_license`?
- `facebook_event_checkin` ➡️ `income`?



The suspect

```
SELECT * FROM person
-- suspect person_id from first witness
WHERE id = 67318
-- suspect person_id from second witness
AND id IN (67318, 28819)
```

The suspect's interview

id	person_id	name	membership_start_date	membership_status
48Z55	67318	Jeremy Bowers	20160101	gold

```
SELECT * FROM interview WHERE person_id = 67318
```

> I was hired by a woman with a lot of money. I don't know her name but I know she's around 5'5" (65") or 5'7" (67"). She has red hair and she drives a Tesla Model S. I know that she attended the SQL Symphony Concert 3 times in December 2017.

- 1️⃣ red hair, Tesla Model S, height 65" or 67"
- 2️⃣ attended SQL Symphony Concert 3 times in December 2017

Filter `drivers_license` by the description

```
SELECT *
FROM drivers_license
WHERE hair_color = 'red'
AND car_make = 'Tesla'
AND car_model = 'Model S'
AND height BETWEEN 65 AND 67
```

id	age	height	eye_color	hair_color	gender	plate_number	car_make	car_model
202298	68	66	green	red	female	500123	Tesla	Model S
291182	65	66	blue	red	female	08CM64	Tesla	Model S
918773	48	65	black	red	female	917UU3	Tesla	Model S

Cross-check with the `person` table

id	age	height	eye_color	hair_color	gender	plate_number	car_make	car_model
202298	68	66	green	red	female	500123	Tesla	Model S
291182	65	66	blue	red	female	08CM64	Tesla	Model S
918773	48	65	black	red	female	917UU3	Tesla	Model S

`WHERE` column `IN` (list of values)

```
SELECT *
FROM person
WHERE license_id IN (918773, 291182, 202298)
```

Cross-check using Subquery

Subquery: a query within a query

The output of one query becomes the input of another query



Clue #1 using Subquery

1️⃣ Query to get the list of `ID`s from `drivers_license` table that match the description

```
-- It must return the column that we want to use in the outer query
SELECT id
FROM drivers_license
WHERE hair_color = 'red'
AND car_make = 'Tesla'
AND car_model = 'Model S'
AND height BETWEEN 65 AND 67
```

id
202298
291182
918773

Clue #1 using Subquery

2️⃣ Insert that query in the `WHERE` clause

`WHERE` column `IN` (subquery returning a list of one column)

```
SELECT *
FROM person
-- Instead of hard-coding the list of IDs
-- where license_id in (918773, 291182, 202298)
-- Use the subquery that returns that list
WHERE license_id IN (
    SELECT id
    FROM drivers_license
    WHERE hair_color = 'red'
    AND car_make = 'Tesla'
    AND car_model = 'Model S'
    AND height BETWEEN 65 AND 67
)
```

Redo the questions with a Subquery

1. Find the interview of the first witness, who lives at the last house on "Northwestern Dr."

- 1️⃣ Query to get the person ID of the last house on Northwestern Dr.
- 2️⃣ Use the query to get the interview

1️⃣ Query to get the person ID of the last house on Northwestern Dr.

```
SELECT id FROM person
WHERE address_street_name = 'Northwestern Dr'
ORDER BY address_number DESC LIMIT 1
```

2️⃣ Use the query to get the interview

```
SELECT * FROM interview
WHERE person_id = (
    SELECT id FROM person
    WHERE address_street_name = 'Northwestern Dr'
    ORDER BY address_number DESC LIMIT 1
    )
```

‼️ A subquery with `LIMIT` must use `=`, not `IN` (in MariaDB)

💻 2. Find the first suspect from person table: membership id starts with "48Z" and license plate includes "H42W"

- Subquery 1: membership id starts with "48Z"
- Subquery 2: license plate includes "H42W"
- Outer query: find the person who matches both subqueries

💻 3. Pull up the interview of the second witness. Her name is Annabel and she lives somewhere on "Franklin Ave"

- Subquery 1: find the person ID of Annabel who lives on Franklin Ave
- Outer query: find the interview of that person ID

Three clues in one query

It works, but can you read it?

```
SELECT * FROM person
-- 7. From first witness:
-- gym membership id starts with "48Z" and license plate includes "H42W"
WHERE id IN (SELECT person_id FROM get_fit_now_member WHERE id LIKE '%48Z%')
AND license_id IN (SELECT id FROM drivers_license WHERE plate_number LIKE '%H42W%')
-- 8. From second witness:
-- the suspect was at the gym between 4 PM and 5 PM
AND id IN (
    SELECT person_id FROM get_fit_now_member
    WHERE id IN (
        SELECT membership_id FROM get_fit_now_check_in
        WHERE check_in_time <= 1700
        AND check_out_time >= 1600
        )
    )
```

Common Table Expressions (CTE)

Temporary named result set that you can reference within another sql statement

`WITH` cte_name `AS` (subquery)

```
-- Define the CTE
WITH suspect_gym AS (
    SELECT person_id FROM get_fit_now_member WHERE id LIKE '48Z%'
)

-- You can reference the CTE like a regular table
SELECT * FROM person WHERE id IN (SELECT * FROM suspect_gym);
```

Rewrite with CTEs

1️⃣ Define multiple CTEs for each clue

`WITH` cte_name1 `AS` (subquery1), cte_name2 `AS` (subquery2), ...

```
WITH
-- membership id starts with "48Z"
suspect_gym AS (SELECT person_id FROM get_fit_now_member WHERE id LIKE '48Z%'),
-- license plate includes "H42W"
suspect_license AS (SELECT id FROM drivers_license WHERE plate_number LIKE '%H42W%'),
-- the suspect was at the gym between 4 PM and 5 PM
suspect_gym_checkin_time AS (
    SELECT person_id FROM get_fit_now_member
    WHERE id IN (
        SELECT membership_id FROM get_fit_now_check_in
        WHERE check_in_time <= 1700
        AND check_out_time >= 1600
        )
    )
```

Rewrite with CTEs (cont.)

2️⃣ Use the CTEs in the main query

```
-- ... WITH block from the previous slide ...

SELECT * FROM person
WHERE id IN (SELECT * FROM suspect_gym)
AND license_id IN (SELECT * FROM suspect_license)
AND id IN (SELECT * FROM suspect_gym_checkin_time)
```

‼️ CTE can be referenced only once

```
WITH suspect_gym AS (
    SELECT person_id FROM get_fit_now_member WHERE id LIKE '48Z%'
)

SELECT * FROM suspect_gym;
SELECT * FROM suspect_gym; -- ERROR: suspect_gym can be used only once
```

Clue #2

1️⃣ ~~red hair, Tesla Model S, height 65" or 67"~~

2️⃣ attended SQL Symphony Concert 3 times in December 2017

Select all attendance of SQL Symphony Concert in 2017

Table: `facebook_event_checkin`

Columns: all

Rows:

- SQL Symphony Concert
- December 2017

```
SELECT *
FROM facebook_event_checkin
WHERE event_name LIKE '%SQL Symphony Concert%'
AND date BETWEEN 20171201 AND 20171231
ORDER BY person_id
```

Aggregate functions

- `count()`
- `sum()`
- `avg()`
- `max()`
- `min()`
- `stddev()` : standard deviation
- `variance()` : variance

  ...

https://dev.mysql.com/doc/refman/8.0/en/aggregate-functions.html

`count(*)` to count the number of rows

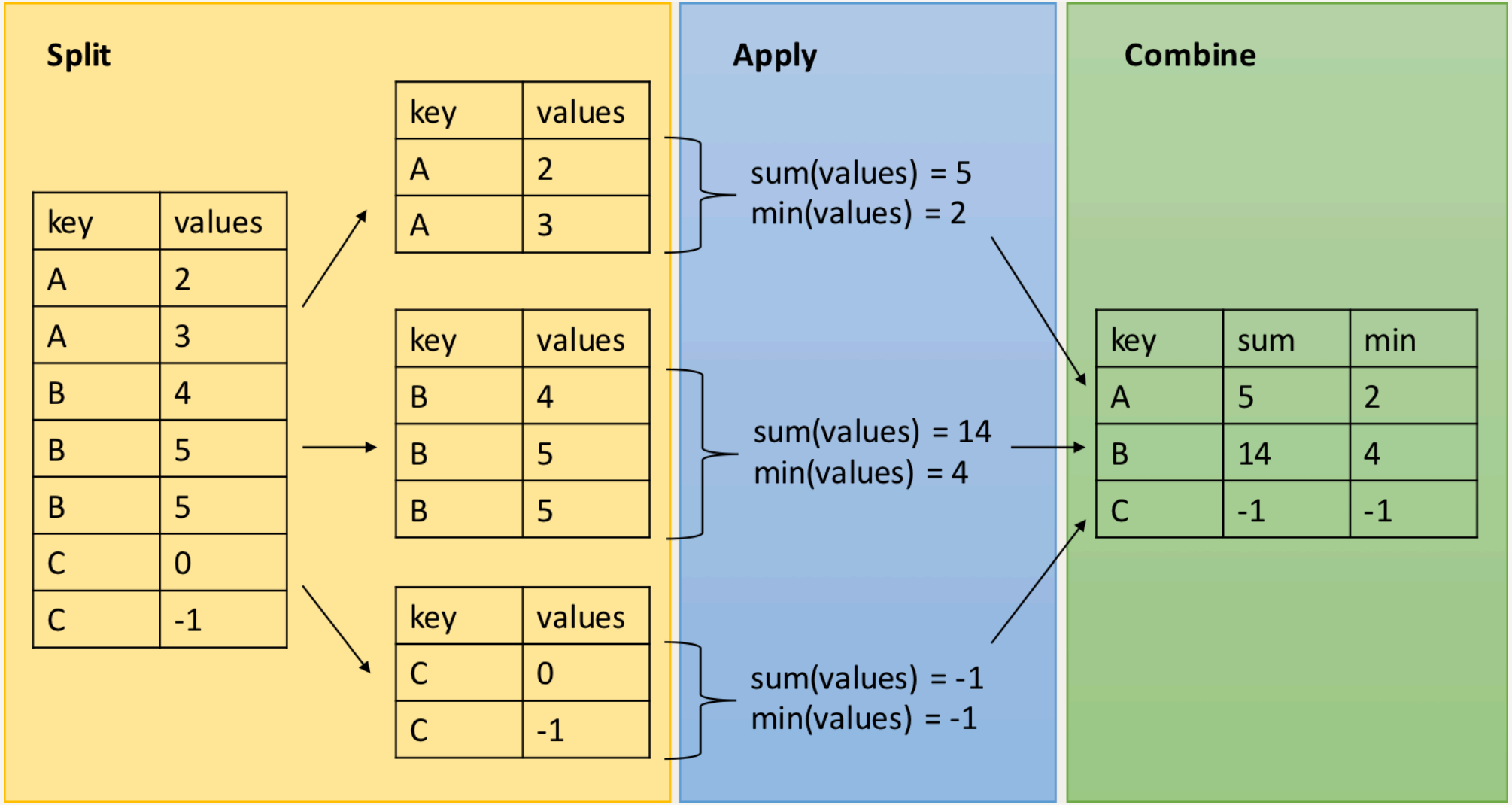
```
SELECT count(*)
FROM facebook_event_checkin
WHERE event_name LIKE '%SQL Symphony Concert%'
AND date BETWEEN 20171201 AND 20171231
ORDER BY person_id
```

`SELECT` column `FROM` table `GROUP BY` column

`group by`:

- group rows that have the same values in specified columns
- "find the number of attendance for each person"

Group rows by columns, apply aggregate functions to each group, and combine the results



`GROUP BY` person to count each person's attendance

- Group rows by person_id
- Apply count function to each group
- Combine the results

`AS` alias: name the new column so later clauses can refer to it

```
SELECT person_id, count(*) AS num_attendance
FROM facebook_event_checkin
WHERE event_name LIKE '%SQL Symphony Concert%'
AND date BETWEEN 20171201 AND 20171231
GROUP BY person_id
```

`WHERE` filters rows, `HAVING` filters groups

```sql
SELECT person_id, count(*) AS num_attendance
FROM facebook_event_checkin

-- filter rows before grouping
WHERE event_name LIKE '%SQL Symphony Concert%'
AND date BETWEEN 20171201 AND 20171231
GROUP BY person_id

-- filter groups after grouping
-- HAVING count(*) = 3 also works
HAVING num_attendance = 3
```

`DISTINCT` to remove duplicates

Q: On how many dates was the SQL Symphony Concert held in 2017?

➡️ Each check-in is one row, so `count(*)` counts people. Count the distinct dates instead.

```
SELECT count(DISTINCT date)
FROM facebook_event_checkin
WHERE event_name LIKE '%SQL Symphony Concert%'
AND date BETWEEN 20171201 AND 20171231
```

💻 From Facebook Event Checkin table

1. What is the earliest check-in date in the table?
2. Which five events had the most check-ins?
3. Who attended three or more different events in December 2017? (several check-ins to the same event count as one)

Putting it All Together

1. Clue #1: People that match the description (`license_id` from `drivers_license`)
2. Clue #2: People that attended SQL Symphony Concert 3 times in 2017 (`person_id` from `facebook_event_checkin`)

Using Subquery

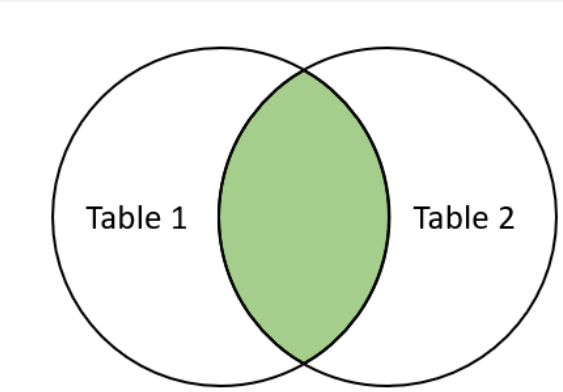
```sql
SELECT *
FROM person
WHERE license_id IN (
    SELECT id
    FROM drivers_license
    WHERE hair_color = 'red'
    AND car_make = 'Tesla'
    AND car_model = 'Model S'
    AND height BETWEEN 65 AND 67
)
AND id IN (
    SELECT person_id
    FROM facebook_event_checkin
    WHERE event_name LIKE '%SQL Symphony Concert%'
    AND date BETWEEN 20171201 AND 20171231
    GROUP BY person_id
    HAVING count(*)=3
)
```

Using Subquery and CTE

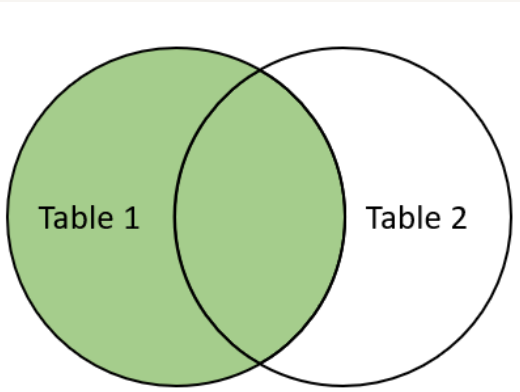
```
WITH
suspect_license AS (
    SELECT id
    FROM drivers_license
    WHERE hair_color = 'red'
    AND car_make = 'Tesla'
    AND car_model = 'Model S'
    AND height BETWEEN 65 AND 67
),
suspect_event AS (
    SELECT person_id
    FROM facebook_event_checkin
    WHERE event_name LIKE '%SQL Symphony Concert%'
    AND date BETWEEN 20171201 AND 20171231
    GROUP BY person_id
    HAVING count(*)=3
)

SELECT *
FROM person
WHERE license_id IN (SELECT * FROM suspect_license)
AND id IN (SELECT * FROM suspect_event)
```

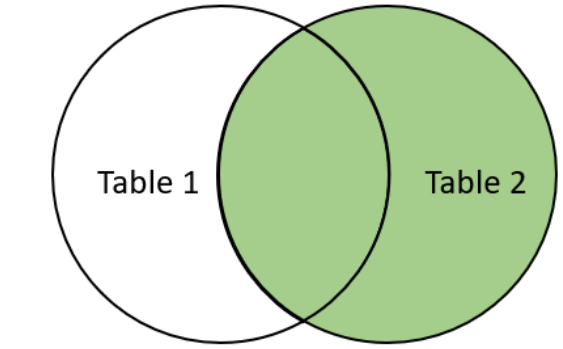
Using `JOIN`



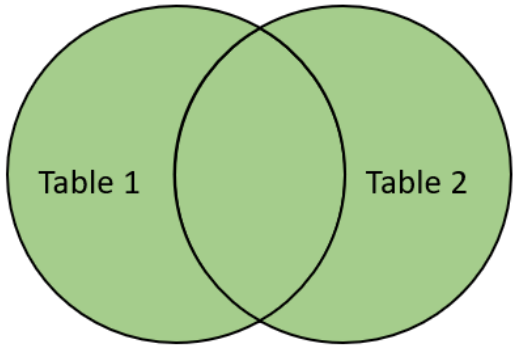
INNER JOIN



LEFT JOIN



RIGHT JOIN



CROSS JOIN

Same two tables, four joins

`person` (on left)

id	name	license_id
1	Alice	101
2	Bob	102
3	Charlie	103

`drivers_license` (on right)

id	hair_color	car_make
101	red	Tesla
102	blue	Ford
104	black	BMW

Match `license_id` in `person` ➡️ `id` in `drivers_license`

- **101, 102** — in both tables
- **103** — only in `person`
- **104** — only in `drivers_license`

The join type decides what happens to 103 and 104.

Inner join: only the matches

name	license_id	id	hair_color
Alice	101	101	red
Bob	102	102	blue

103 and 104 are gone: no match on the other side.

Left and right join: keep one side whole

Left join: keep all of `person`

name	license_id	id	hair_color
Alice	101	101	red
Bob	102	102	blue
Charlie	103	NULL	NULL

Right join: keep all of `drivers_license`

name	license_id	id	hair_color
Alice	101	101	red
Bob	102	102	blue
NULL	NULL	104	black

Cross join: every combination

Every row is paired with every row.

3 persons × 3 licenses = **9 rows**

name	license_id	id	hair_color
Alice	101	101	red
Alice	101	102	blue
Alice	101	104	black
Bob	102	101	red
Bob	102	102	blue
Bob	102	104	black
Charlie	103	101	red
Charlie	103	102	blue
Charlie	103	104	black

Writing a `JOIN ON`: which tables, on which columns?

`SELECT` * `FROM` table1 `JOIN` table2 `ON` table1.column = table2.column

1. Which tables? `person` and `drivers_license`
2. On which columns? `person.license_id` = `drivers_license.id`

```
-- id exists in both tables, so say which one: table.column
SELECT person.id, name, hair_color
-- 1. tables to join
FROM person JOIN drivers_license
-- 2. join condition
ON person.license_id = drivers_license.id
```

`JOIN` alone = inner join: only rows with a match on both sides are kept.

🤔 On which columns?

1. `person JOIN drivers_license ON person.id = drivers_license.id`. Is it right?
2. Join `person` and `facebook_event_checkin` on?

Once joined, it's one table

`FROM ... JOIN ... ON` produces a single table. `SELECT` and `WHERE` work on that table, using columns from either side.

```
SELECT *
-- one table with all columns from person and drivers_license
FROM person JOIN drivers_license
ON person.license_id = drivers_license.id
-- filter it like any table: these columns come from drivers_license
WHERE hair_color = 'red'
AND car_make = 'Tesla'
AND car_model = 'Model S'
AND height BETWEEN 65 AND 67
```

Clue #1 again, with no subquery.

`JOIN` or subquery? Depends on what you need back

- `JOIN drivers_license`: if its columns must appear **in the result** (hair, car, etc.)
- Subquery on `facebook_event_checkin`: only decides **who qualifies**; none of its columns are needed back

```sql
SELECT *
FROM person JOIN drivers_license
ON person.license_id = drivers_license.id
WHERE hair_color = 'red' AND car_make = 'Tesla'
    AND car_model = 'Model S'
    AND height BETWEEN 65 AND 67
AND person.id IN (
    SELECT person_id
    FROM facebook_event_checkin
    WHERE event_name LIKE '%SQL Symphony Concert%'
    AND date BETWEEN 20171201 AND 20171231
    GROUP BY person_id
    HAVING count(*) = 3);
```



`JOIN` to show columns from another table

1. Show the second witness's name next to her interview transcript. (Annabel, lives on "Franklin Ave"; tables: `person`, `interview`)
2. The suspect said "a woman with a lot of money". Start from the Clue #1 join (`person JOIN drivers_license`, filtered by the description) and add each person's `annual_income` to the result. (tables: `person`, `drivers_license`, `income`)

Other `JOIN` syntaxes

`JOIN USING` and `NATURAL JOIN`: Shortcuts when the join column has the **same name** in both tables (`person.ssn` = `income.ssn`)

```
-- USING: name the shared column once
SELECT * FROM person JOIN income USING (ssn);
-- NATURAL JOIN: join on ALL columns with the same name (here, only ssn)
SELECT * FROM person NATURAL JOIN income;
-- ⚠ NATURAL JOIN silently picks the wrong column when names coincide
SELECT * FROM person NATURAL JOIN drivers_license;  -- joins on id, which is wrong
```

Older style: tables in `FROM`, join condition in `WHERE`. Same result as `JOIN ON`.

```
SELECT * FROM person, drivers_license
WHERE person.license_id = drivers_license.id
```

Prefer `JOIN ON`: the condition is explicit, so it's readable and it can't be wrong by accident.

`LEFT JOIN` and `RIGHT JOIN`

`LEFT JOIN`:

Table on the left: all rows

Table on the right: matching rows, `NULL` where there's no match

```
SELECT * FROM person LEFT JOIN get_fit_now_member
ON person.id = get_fit_now_member.person_id
```

`RIGHT JOIN`:

Table on the left: matching rows, `NULL` where there's no match

Table on the right: all rows

```
SELECT * FROM person RIGHT JOIN get_fit_now_member
ON person.id = get_fit_now_member.person_id
```

Wrap up

- Subquery: query within a query
- Common Table Expressions (CTE): temporary named result set
- Aggregate functions: `count()`, `sum()`, `avg()`, `max()`, `min()`, ...
- `GROUP BY`: group rows by column(s)
- `HAVING`: filter groups after grouping
- `JOIN`: combine data from multiple tables